SQL Queries: Evidence

Mini Project 1: Northwest Architecture Studio · Marcus Weeks (Architect / CAD Designer build)

Run 29 September 2026 · readable copy 7 October 2026

Every query below was run on OneCompiler (MySQL) on 29/09/2026, on the database built from the brief's ER diagram (page 8). Each picture shows the query and its result, cut from the full OneCompiler screenshot so the text can be read. The full screenshots are in the Evidence folder on GitHub.

Queries 5, 6 and 7 in the brief ask about build number 1. This database only holds the CAD build's parts so far (build 3), so they are run on build 3. They work the same way on build 1 once those parts are added.

Part A: Reading single tables

Query 1: List every role

```
SELECT * FROM EmployeeRoles;
```

Shows every column for all 4 roles. The * means "every column".

Query (bottom of the code, left side of OneCompiler):

```
92    -- Query 1: List every role
93    SELECT * FROM EmployeeRoles;
```

Result (Output panel):

Output:

RoleID	RoleName	StaffCount	Tasks	PerformanceNeed	BudgetPerUnit
1	Administrative Assistant	3	Email, documents, spreadsheets, web systems, online meetings, and basic customer records	Basic office hardware workstation	950.00
2	Project Manager	2	Reports, meetings, project planning, reviewing drawings, dashboards, and client communication	Business workstation with stronger display/workflow setup	1300.00
3	Architect / CAD Designer	5	CAD drawings, 2D drafting, 3D modelling, large design files, multiple monitors, and design reviews	CAD workstation with dedicated GPU and suitable display	2500.00
4	3D Visualisation Specialist	2	Rendering, 3D visualisation, large models, high-resolution textures, media output, and client visual presentations	High-performance rendering/visualisation workstation	3750.00

Query 1 result

Query 2: Just the names and budgets

```
SELECT RoleName, BudgetPerUnit FROM EmployeeRoles;
```

Shows only the 2 columns asked for: the role name and its budget per workstation.

Query (bottom of the code, left side of OneCompiler):

```
92    -- Query 2: Just the names and budgets
93    SELECT RoleName, BudgetPerUnit FROM EmployeeRoles;
```

Result (Output panel):

Output:

```
+-----------------------------+---------------+
| RoleName                    | BudgetPerUnit |
+-----------------------------+---------------+
| Administrative Assistant    |        950.00 |
| Project Manager             |       1300.00 |
| Architect / CAD Designer    |       2500.00 |
| 3D Visualisation Specialist |       3750.00 |
+-----------------------------+---------------+
```

Query 2 result

Query 3: The higher-budget roles

```
SELECT RoleName, BudgetPerUnit FROM EmployeeRoles WHERE BudgetPerUnit > 1000;
```

`WHERE` keeps only the rows that match. 3 roles have a budget over £1,000. The Administrative Assistant (£950) drops out.

Query (bottom of the code, left side of OneCompiler):

```
92    -- Query 3: The higher-budget roles (over £1000)
93    SELECT RoleName, BudgetPerUnit FROM EmployeeRoles WHERE BudgetPerUnit > 1000;
```

Result (Output panel):

Output:

```
+-----------------------------+---------------+
| RoleName                    | BudgetPerUnit |
+-----------------------------+---------------+
| Project Manager             |       1300.00 |
| Architect / CAD Designer    |       2500.00 |
| 3D Visualisation Specialist |       3750.00 |
+-----------------------------+---------------+
```

Query 3 result

Query 4: Roles by team size

```
SELECT RoleName, StaffCount FROM EmployeeRoles ORDER BY StaffCount DESC;
```

`ORDER BY ... DESC` sorts from biggest to smallest. The CAD Designers come first with 5 staff.

Query (bottom of the code, left side of OneCompiler):

```
92    -- Query 4: Roles by team size, largest first
93    SELECT RoleName, StaffCount FROM EmployeeRoles ORDER BY StaffCount DESC;
```

Result (Output panel):

Output:

```
+-----------------------------+------------+
| RoleName                    | StaffCount |
+-----------------------------+------------+
| Architect / CAD Designer    |          5 |
| Administrative Assistant    |          3 |
| Project Manager             |          2 |
| 3D Visualisation Specialist |          2 |
+-----------------------------+------------+
```

Query 4 result

Query 5: Find one build’s items

```sql
SELECT * FROM BuildItems WHERE BuildID = 3;
```

Lists all 14 items in the CAD build, with category, name, specification and cost.

Query (bottom of the code, left side of OneCompiler):

```sql
92   -- Query 5: Every item in my build (build 3)
93   SELECT * FROM BuildItems WHERE BuildID = 3;
```

Result (Output panel):

Output:

ItemID	BuildID	HardwareCategory	ItemName	Specification	Cost
301	3	CPU	Intel Core i5 14400F	10 cores (6P+4E), 16 threads, up to 4.7GHz, LGA1700, 65W, cooler included, no built-in graphics	139.99
302	3	Motherboard	MSI B760 GAMING PLUS WIFI DDR5 ATX	LGA1700, Intel B760, 4x DDR5 slots, PCIe 4.0 x16, 2x M.2, WiFi 6E, 2.5Gb ethernet	95.99
303	3	RAM	Patriot Viper Venom 32GB (2x16GB)	32GB (2x16GB) DDR5-6000 CL30	399.95
304	3	Storage	Kingston NV3 1TB	1TB M.2-2280 PCIe 4.0 x4 NVMe SSD	128.99
305	3	GPU	Gigabyte WINDFORCE OC GeForce RTX 3060 12GB	GeForce RTX 3060, 12GB GDDR6	422.15
306	3	PSU	be quiet! Pure Power 12 650W	650W, 80+ Gold, non-modular	68.33
307	3	Case/Cooling	Montech X3 Mesh ATX Mid Tower	ATX mid tower, mesh front, tempered glass side, 6 fans pre-installed	29.99
308	3	Monitor	KTC 27-inch QHD IPS Monitor	27-inch, 2560x1440 QHD, IPS, 100Hz, HDMI + DisplayPort	234.78
309	3	Monitor	KTC 27-inch QHD IPS Monitor	Second identical screen for dual-monitor setup	234.78
310	3	Keyboard	Logitech K120	UK layout, wired USB	10.99
311	3	Mouse	Lenovo ThinkPad Precision Wireless Optical Mouse	Wireless optical mouse	36.11
312	3	Webcam/Headset	Logitech C270 Webcam	720p USB webcam	16.99
313	3	Webcam/Headset	Razer Kraken 7.1 Channel Closed Headset	Closed-back wired headset with microphone	21.98
314	3	Network/Cables/Adapters	Nixsto Cat6 Ethernet Cable	Cat6 ethernet patch cable, 2m, RJ45	3.48

Query 5 result

Query 6: Count the items in a build

```
SELECT COUNT(*) AS ItemCount FROM BuildItems WHERE BuildID = 3;
```

`COUNT(*)` counts the rows. The CAD build has 14 items. `AS` gives the answer column a clear name.

Query (bottom of the code, left side of OneCompiler):

```
92    -- Query 6: Count the items in my build
93    SELECT COUNT(*) AS ItemCount FROM BuildItems WHERE BuildID = 3;
```

Result (Output panel):

Output:

```
+-----------+
| ItemCount |
+-----------+
|        14 |
+-----------+
```

Query 6 result

Part B: Totals and summaries

Query 7: Total cost of one build

```
SELECT SUM(Cost) AS TotalCost FROM BuildItems WHERE BuildID = 3;
```

SUM adds up the Cost column. One CAD workstation costs £1,844.50.

Query (bottom of the code, left side of OneCompiler):

```
92    -- Query 7: Total cost of my build
93    SELECT SUM(Cost) AS TotalCost FROM BuildItems WHERE BuildID = 3;
```

Result (Output panel):

Output:

```
+-----------+
| TotalCost |
+-----------+
|   1844.50 |
+-----------+
```

Query 7 result

Query 8: Cost of every build

```
SELECT BuildID, SUM(Cost) AS TotalCost FROM BuildItems GROUP BY BuildID ORDER BY TotalCost DESC;
```

`GROUP BY` gives one total per build, and `ORDER BY ... DESC` puts the most expensive first. Only build 3 has parts so far, so it shows one row.

Query (bottom of the code, left side of OneCompiler):

```
92    -- Query 8: Cost of every build, most expensive first
93    SELECT BuildID, SUM(Cost) AS TotalCost FROM BuildItems GROUP BY BuildID ORDER BY TotalCost DESC;
```

Result (Output panel):

Output:

```
+---------+-----------+
| BuildID | TotalCost |
+---------+-----------+
|       3 |   1844.50 |
+---------+-----------+
```

Query 8 result

Part C: Across tables with JOINs

Query 9: Which role is each build for?

```
SELECT b.BuildName, r.RoleName
FROM WorkstationBuilds b
JOIN EmployeeRoles r ON b.RoleID = r.RoleID;
```

The build names and role names live in different tables. `JOIN` matches them up using the RoleID they share. `b` and `r` are short nicknames for the 2 tables.

Query (bottom of the code, left side of OneCompiler):

```
92   -- Query 9: Which role is each build for?
93   SELECT b.BuildName, r.RoleName
94   FROM WorkstationBuilds b
95   JOIN EmployeeRoles r ON b.RoleID = r.RoleID;
```

Result (Output panel):

Output:

```
+------------------------+-----------------------------+
| BuildName              | RoleName                    |
+------------------------+-----------------------------+
| Admin Build            | Administrative Assistant    |
| Project Manager Build  | Project Manager             |
| CAD Build              | Architect / CAD Designer    |
| 3D Visualisation Build | 3D Visualisation Specialist |
+------------------------+-----------------------------+
```

Query 9 result

Query 10: Items with their build and role

```
SELECT i.ItemName, b.BuildName, r.RoleName
FROM BuildItems i
JOIN WorkstationBuilds b ON i.BuildID = b.BuildID
JOIN EmployeeRoles r ON b.RoleID = r.RoleID;
```

Links 3 tables. Each item points to its build, and each build points to its role, so every part shows which build and which role it belongs to.

Query (bottom of the code, left side of OneCompiler):

```
92   -- Query 10: Items with their build and role
93   SELECT i.ItemName, b.BuildName, r.RoleName
94   FROM BuildItems i
95   JOIN WorkstationBuilds b ON i.BuildID = b.BuildID
96   JOIN EmployeeRoles r ON b.RoleID = r.RoleID;
```

Result (Output panel):

Output:

ItemName	BuildName	RoleName
Intel Core i5 14400F	CAD Build	Architect / CAD Designer
MSI B760 GAMING PLUS WIFI DDR5 ATX	CAD Build	Architect / CAD Designer
Patriot Viper Venom 32GB (2x16GB)	CAD Build	Architect / CAD Designer
Kingston NV3 1TB	CAD Build	Architect / CAD Designer
Gigabyte WINDFORCE OC GeForce RTX 3060 12GB	CAD Build	Architect / CAD Designer
be quiet! Pure Power 12 650W	CAD Build	Architect / CAD Designer
Montech X3 Mesh ATX Mid Tower	CAD Build	Architect / CAD Designer
KTC 27-inch QHD IPS Monitor	CAD Build	Architect / CAD Designer
KTC 27-inch QHD IPS Monitor	CAD Build	Architect / CAD Designer
Logitech K120	CAD Build	Architect / CAD Designer
Lenovo ThinkPad Precision Wireless Optical Mouse	CAD Build	Architect / CAD Designer
Logitech C270 Webcam	CAD Build	Architect / CAD Designer
Razer Kraken 7.1 Channel Closed Headset	CAD Build	Architect / CAD Designer
Nixsto Cat6 Ethernet Cable	CAD Build	Architect / CAD Designer

Query 10 result

Query 11: Spend per role

```
SELECT r.RoleName, SUM(i.Cost) AS TotalSpend
FROM BuildItems i
JOIN WorkstationBuilds b ON i.BuildID = b.BuildID
JOIN EmployeeRoles r ON b.RoleID = r.RoleID
GROUP BY r.RoleName;
```

Combines a JOIN with SUM and GROUP BY to total the hardware cost for each role. The CAD role comes to £1,844.50 per workstation.

Query (bottom of the code, left side of OneCompiler):

```
92   -- Query 11: Spend per role
93   SELECT r.RoleName, SUM(i.Cost) AS TotalSpend
94   FROM BuildItems i
95   JOIN WorkstationBuilds b ON i.BuildID = b.BuildID
96   JOIN EmployeeRoles r ON b.RoleID = r.RoleID
97   GROUP BY r.RoleName;
```

Result (Output panel):

Output:

```
+--------------------------+------------+
| RoleName                 | TotalSpend |
+--------------------------+------------+
| Architect / CAD Designer |    1844.50 |
+--------------------------+------------+
```

Query 11 result

Extra: role totals against the budget

```
SELECT r.RoleName,
       r.StaffCount,
       SUM(i.Cost) AS CostPerPC,
       SUM(i.Cost) * r.StaffCount AS RoleTotal,
       r.BudgetPerUnit * r.StaffCount AS RoleBudget
FROM BuildItems i
JOIN WorkstationBuilds b ON i.BuildID = b.BuildID
JOIN EmployeeRoles r ON b.RoleID = r.RoleID
GROUP BY r.RoleID, r.RoleName, r.StaffCount, r.BudgetPerUnit;
```

Multiplies the cost of one workstation by the number of staff. 5 CAD workstations cost £9,222.50 against a role budget of £12,500.

Query (bottom of the code, left side of OneCompiler):

```
92   -- Role totals vs budget
93   SELECT r.RoleName,
94          r.StaffCount,
95          SUM(i.Cost) AS CostPerPC,
96          SUM(i.Cost) * r.StaffCount AS RoleTotal,
97          r.BudgetPerUnit * r.StaffCount AS RoleBudget
98   FROM BuildItems i
99   JOIN WorkstationBuilds b ON i.BuildID = b.BuildID
100  JOIN EmployeeRoles r ON b.RoleID = r.RoleID
101  GROUP BY r.RoleID, r.RoleName, r.StaffCount, r.BudgetPerUnit;
```

Result (Output panel):

Output:

```
+--------------------------+------------+-----------+-----------+------------+
| RoleName                 | StaffCount | CostPerPC | RoleTotal | RoleBudget |
+--------------------------+------------+-----------+-----------+------------+
| Architect / CAD Designer |          5 |   1844.50 |   9222.50 |   12500.00 |
+--------------------------+------------+-----------+-----------+------------+
```

Role totals result

Checking the other tables

The brief asks for every table to be queried at least once.

```
SELECT * FROM WorkstationBuilds;
```

Query (bottom of the code, left side of OneCompiler):

```
92    -- Check: WorkstationBuilds
93    SELECT * FROM WorkstationBuilds;
```

Result (Output panel):

Output:

```
+---------+--------+-----------------------+----------------+
| BuildID | RoleID | BuildName             | Source         |
+---------+--------+-----------------------+----------------+
|       1 |      1 | Admin Build           | BuildCores.com |
|       2 |      2 | Project Manager Build | BuildCores.com |
|       3 |      3 | CAD Build             | BuildCores.com |
|       4 |      4 | 3D Visualisation Build | BuildCores.com |
+---------+--------+-----------------------+----------------+
```

WorkstationBuilds

```
SELECT * FROM Recommendations;
```

The Justification, Compromise and EthicalNotes columns hold full paragraphs. The result is wrapped onto several lines so every word shows, which is why the table lines break across the screen. All 6 columns are there: RecommendationID 3, RoleID 3, the three paragraphs, and Approved = 1 (yes).

Query (bottom of the code, left side of OneCompiler):

```
92    -- Check: Recommendations
93    SELECT * FROM Recommendations;
```

Result (Output panel):

Output:

```
+------------------+--------+-------------------
```

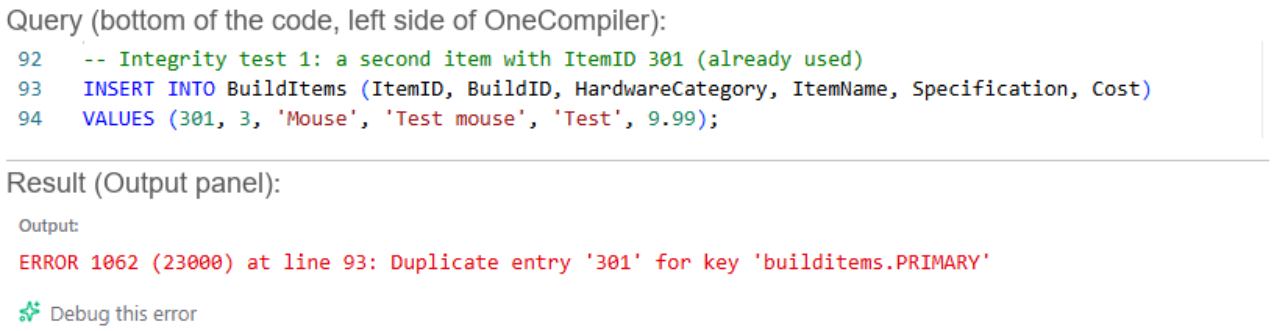
Data integrity: how the database stops bad data

The brief's data integrity rules (page 9) are built into the tables themselves, so the database refuses bad data instead of storing it. Each rule below was tested on OneCompiler by trying to add a bad row on purpose.

Unique IDs. Every table has a `PRIMARY KEY`: RoleID, BuildID, ItemID and RecommendationID. No two rows can share one, so two parts can never get mixed up. Adding a second item with ItemID 301 was refused.

```
INSERT INTO BuildItems (ItemID, BuildID, HardwareCategory, ItemName,
Specification, Cost)
VALUES (301, 3, 'Mouse', 'Test mouse', 'Test', 9.99);
```

Result: `ERROR 1062: Duplicate entry '301' for key 'builditems.PRIMARY'`



Integrity test 1

Links must point to something real. Each `FOREIGN KEY` checks the other table. A part has to belong to a build that exists, and every build and recommendation has to belong to a real role. Adding a part to build 9, which doesn't exist, was refused.

```
INSERT INTO BuildItems (ItemID, BuildID, HardwareCategory, ItemName, Specification, Cost)
VALUES (999, 9, 'Mouse', 'Test mouse', 'Test', 9.99);
```

Result: `ERROR 1452: Cannot add or update a child row: a foreign key constraint fails`



Integrity test 2

Required fields can't be blank. Every column is `NOT NULL`, which covers the ones the brief names: role name, hardware category, item name, cost and justification. Adding a part with no name was refused.

```
INSERT INTO BuildItems (ItemID, BuildID, HardwareCategory, ItemName,
Specification, Cost)
VALUES (998, 3, 'Mouse', NULL, 'Test', 9.99);
```

Result: `ERROR 1048: Column 'ItemName' cannot be null`

Query (bottom of the code, left side of OneCompiler):

```
92   -- Integrity test 3: an item with no name (NULL)
93   INSERT INTO BuildItems (ItemID, BuildID, HardwareCategory, ItemName, Specification, Cost)
94   VALUES (998, 3, 'Mouse', NULL, 'Test', 9.99);
```

Result (Output panel):

Output:

```
ERROR 1048 (23000) at line 93: Column 'ItemName' cannot be null
```

Debug this error

Integrity test 3

Costs must be numbers. Cost is `DECIMAL(8,2)`, which stores money to the exact penny, so `SUM` can add it up. A cost written as a word was refused.

```
INSERT INTO BuildItems (ItemID, BuildID, HardwareCategory, ItemName,
Specification, Cost)
VALUES (997, 3, 'Mouse', 'Test mouse', 'Test', 'cheap');
```

Result: `ERROR 1366: Incorrect decimal value: 'cheap' for column 'Cost'`

Query (bottom of the code, left side of OneCompiler):

```
92   -- Integrity test 4: a cost written as a word ('cheap')
93   INSERT INTO BuildItems (ItemID, BuildID, HardwareCategory, ItemName, Specification, Cost)
94   VALUES (997, 3, 'Mouse', 'Test mouse', 'Test', 'cheap');
```

Result (Output panel):

Output:

```
ERROR 1366 (HY000) at line 93: Incorrect decimal value: 'cheap' for column 'Cost' at row 1
```

Debug this error

Integrity test 4

Consistent category names. Every part uses one of the same names, following the brief's list: CPU, Motherboard, RAM, Storage, GPU, PSU, Case/Cooling, Monitor, Keyboard, Mouse, Webcam/Headset and Network/Cables/Adapters. A misspelling like "Moniter" would be counted as its own category and throw out `COUNT` and `GROUP BY` totals. The database doesn't block this one by itself, so it depends on everyone in the team using the same list.

Team numbering. Each member's ItemIDs start from their build number: Admin 101 onwards, Project Manager 201, CAD 301, 3D Visualisation 401. Everyone can add parts to the same table without their IDs clashing.

Checklist (brief pages 9 to 12)

- ☒ Every table queried: EmployeeRoles, WorkstationBuilds, BuildItems, Recommendations
- ☒ Used at least once: SELECT, WHERE, ORDER BY, COUNT, SUM, GROUP BY, JOIN
- ☒ A screenshot of every result
- ☒ Each query explained
- ☒ Data integrity explained, with tested examples (unique IDs, valid links, required fields, numeric costs, consistent categories)